SIMATS ENGINEERING

Department of Artificial Intelligence and Data Science

JAVA PROGRAMMING – CO1

ASSIGNMENT 1

Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System

CO1: Java fundamentals and object-oriented programming – data types, operators, control statements, arrays, classes and objects, constructors, static members, inheritance, method overriding, polymorphism, abstract classes, interfaces, packages, access modifiers and encapsulation.

Student Details	Information	Student Details	Information
Student Name	P. Dada Kalander	Register Number	192424272
Department / Section	B.Tech AI&DS	Academic Year	2026–2027
Faculty Name	G. Jessy	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	11/11/2026	GitHub Repository	CSA09-192424272-Assignment1
Programming Language	Java	JDK Version	17 or later

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max Marks	Marks Awarded	Remarks
1	Problem Analysis & Objects	4		
2	Data Representation	3		
3	Control Logic & Pseudocode	4		
4	Classes, Objects & Constructors	4		
5	Static Members & Object Life Cycle	3		
6	Inheritance Hierarchy	4		
7	Method Overriding	4		
8	Polymorphism	4		
9	Abstraction	4		
10	Packages & Encapsulation	3		
11	Implementation, Testing & Defence	5		
12	Day-4 Design Challenge	8		
	TOTAL	50		

OVERALL REMARKS

Student Declaration	Faculty Evaluation
I declare that the submitted design, source code and test evidence have been prepared and executed by me. I will be able to explain the implementation during verification.	

Student Signature	Faculty Signature	Program Director
P. Dada Kalander		

JAVA PROGRAMMING

ASSIGNMENT 1 – CO1

Assignment Title: Design, Implement and Validate an Object-Oriented Model for a Campus Smart-Parking System (Total Marks – 50).

Scenario summary: The campus operates a multi-level parking facility for two-wheelers, cars and electric vehicles. The system allocates slots, issues unique tickets, records entry, calculates exit fees, supports monthly passes and is later extended to e-bikes, delivery vans, one-day passes and floors with unequal slot counts.

Mandatory Execution and GitHub Submission

- Create a GitHub repository named CSA09-192424272-Assignment1 and share it with the course faculty.
- Maintain one folder per task, with Java source files, input data and output evidence.
- Include a README.md with student details, JDK version, compile/run commands and task summaries.
- Commit progressively, with at least one meaningful commit for every task. Add the repository URL to the cover page before final submission.
- Execute all supplied Complete the Code programs. The included outputs are reference captures from the current Java runtime; reproduce them on the course lab machine.

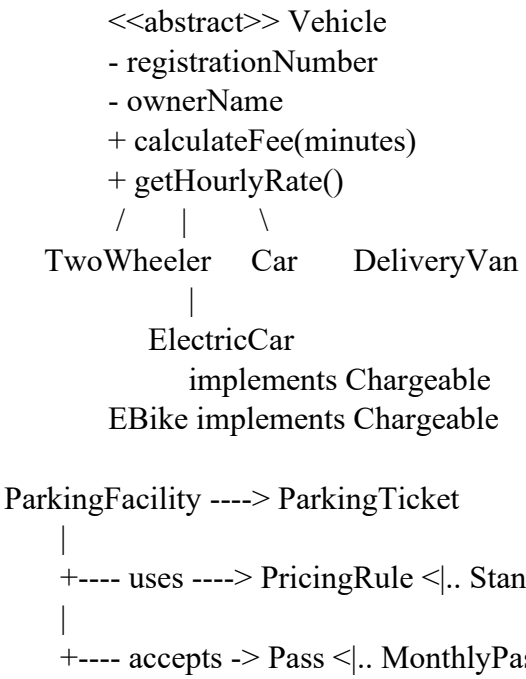
A valid pass waives the parking fee. Charging energy is separately modelled as a Chargeable capability; in this demonstration a valid pass waives the complete parking transaction charge.

GitHub URL to be completed by the student: https://github.com/Kalanderp/CSA09_Java_192424272/Assignments/CO1

System-wide design overview

Layer / type	Responsibility
parking.model	Vehicle hierarchy, ticket, chargeable capability and pass types.
parking.pricing	PricingRule abstraction and standard pricing implementation.
parking.service	ParkingFacility allocation, active-ticket registry, exit processing and occupancy.
parking.app	Runnable demonstrations and complete-code exercises.

Class relationship overview (UML-style)



The implementation uses a jagged boolean array for unequal floor capacities and a HashMap keyed by registration number for active tickets. This allows a slot to be located by scanning the physical layout and an exit ticket to be retrieved by key.

TASK 1 – Understand the Problem and Identify the Objects (4 Marks)

The system receives vehicle details, vehicle category, owner details, pass status, entry time, exit duration and (for charging-capable vehicles) energy consumed. It must return a slot position and ticket at entry, then calculate a fee, release the slot and update occupancy at exit.

Candidate object / class	Main attributes	Responsibilities and justification
ParkingFacility	Floors, slots, occupancy, active tickets	Owns the physical parking layout and coordinates entry/exit. It prevents double allocation and centralises facility state.
Vehicle (abstract)	Registration number, owner	Represents shared identity and defines the fee contract. Abstract because a generic vehicle has no single rate.
TwoWheeler / Car / ElectricCar	Type-specific rate or charging capability	Represent different fee behaviours. ElectricCar extends Car and implements Chargeable.
ParkingTicket	Unique number, vehicle, floor, slot, entry time	Records one parking transaction and provides an auditable link between vehicle and allocated slot.
Pass / MonthlyPass / OneDayPass	Validity period	Separates free-parking eligibility from vehicle type and permits new pass products to be added.
PricingRule	Pricing policy	Keeps fee policy replaceable rather than embedding every tariff in ParkingFacility.
Chargeable	Charging operation	Models an optional capability independently of the vehicle inheritance tree.

Inputs and outputs

- Inputs: registration number, owner, vehicle subtype, available layout, entry/exit time or duration, pass validity and charging energy.
- Outputs: floor/slot coordinates, unique ticket ID, fee breakdown, occupancy and validation messages.

Business rules

- A slot is allocated only when it is free and suitable for the vehicle.
- An active registration number cannot receive a second ticket.
- Ticket IDs are generated monotonically and are not reused during the process lifetime.
- A release request must reference a valid, occupied slot.
- Duration and charging energy cannot be negative.
- A valid pass is checked against the exit date; invalid or expired passes do not waive the fee.
- A vehicle with no available slot is not admitted and no ticket is created.

Assumptions

The first available slot in floor-major order is acceptable. The base assignment's example rates are illustrative rather than an official campus tariff. Times are represented as local time values; the demonstration accepts duration and entry hour as inputs to keep fee calculations deterministic. For a production deployment, entry and exit timestamps should be stored as LocalDateTime and elapsed minutes derived using Duration.

Alternative concept: objects versus parallel arrays

With Vehicle objects, registration number, owner and behaviour are grouped in one object. A parallel-array design stores regNo[], type[] and entryTime[] at matching indexes. Both approaches require a linear scan if no index is maintained, so registration search is O(n). The object design uses O(n) references plus per-object state; parallel arrays store the same logical information but require index synchronisation. Adding an attribute requires one more array and updating every insertion, deletion and sorting path. With objects, the attribute is added to Vehicle or a subtype, reducing structural coupling.

Criterion	Vehicle objects	Parallel arrays
Find registration	O(n) linear scan; one object carries all fields	O(n) scan; matching indexes must stay aligned
Add a new attribute	Add field/getter or subtype field	Add another array and update every mutation
Space	O(n) objects and references	O(n) values across parallel arrays
Reliability	Encapsulation can protect invariants	Index mismatch can associate wrong values

TASK 2 – Choose the Data Representation (3 Marks)

Field	Java representation	Reason
Floor and slot index	int	Zero-based array indexes; validate bounds before access.
Registration number	String	Variable-length alphanumeric identifier; normalise to uppercase.
Entry time	LocalDateTime	Stores date and time without lossy text parsing.
Duration	int minutes or long seconds	Integral elapsed unit avoids floating-point duration errors.
Fee	long paise in production; double only for display/demo	Integer paise avoids decimal rounding error in money calculations.
Pass validity	LocalDate	Date-only comparison for day/month pass validity.
Charging energy	double kWh	Measured quantity can be fractional; validate non-negative values.

Array structure

The original facility can have floors with different capacities. Therefore, String[][] or boolean[][] is implemented as a jagged array: each floor owns an array whose length is configured independently. A rectangular array wastes positions when floor capacities differ and incorrectly assumes a uniform number of slots.

Type conversion and precision

```
int minutes = 61;
int wrongHours = minutes / 60;       // 1; truncates the fractional hour
int billableHours = (minutes + 59) / 60; // 2; ceiling using integers
double wrong = (double)(7 / 2);      // 3.0: division already truncated
double right = (double)7 / 2;        // 3.5
```

The fee algorithm uses integer ceiling division, (minutes + 59) / 60, and converts to a monetary display only after the exact amount is calculated. For money, storing paise in long is safer than repeated double addition.

Input method

Scanner is suitable for a small console assignment because it reads strings, integers and decimals interactively. In a larger service, command-line arguments or validated request objects would separate input parsing from business logic.

Alternative concept: jagged, flat and list representations

Representation	Slot lookup	Space and unequal floors
Jagged array	O(1) direct access when floor and slot are known; O(F×S) worst-case scan	O(total slots); naturally supports different capacities.
Flat one-dimensional array	O(1) only with index=floor×slotsPerFloor+slot	Works cleanly only for equal capacities; variable floor lengths need prefix offsets.
ArrayList<String[]>	O(1) indexed access by floor and slot on average	O(total slots) plus list overhead; supports variable floor sizes and resizing.

The chosen jagged array is simplest and direct for this scenario. A flat array would require an offset table when each floor has a different slot count.

TASK 3 – Design the Control Logic (4 Marks)

Allocation pseudocode

```
ALGORITHM AllocateSlot(vehicleNo)
  FOR each floor f from 0 to floorCount - 1
    FOR each slot s from 0 to slotCount[f] - 1
      IF slot[f][s] is occupied
        CONTINUE
      END IF
      slot[f][s] <- vehicleNo
      RETURN (f, s)
    END FOR
  END FOR
  RETURN NULL
END ALGORITHM
```

Fee computation pseudocode

```
ALGORITHM CalculateFee(vehicle, minutes, entryHour, pass)
  IF minutes < 0 THEN
    RETURN INVALID
  END IF
  IF pass is valid on exitDate THEN
    RETURN 0
  END IF
  hours <- (minutes + 59) DIV 60
  IF vehicle is DeliveryVan AND 9 <= entryHour < 17 THEN
    rate <- 2 * vehicle.hourlyRate
  ELSE
    rate <- vehicle.hourlyRate
  END IF
  RETURN hours * rate
END ALGORITHM
```

The nested loops stop immediately after the first free slot by return. continue skips occupied positions. break could be used to leave the inner loop after a match, but return is clearer because the method must return the selected coordinates. If no slot is found, null indicates a full facility.

Completed Code – Task 3

The source file Task03ParkingFacility.java completes both TODO sections without modifying the supplied main() logic. It checks floor and slot bounds before releasing a slot.

```
Expected output:
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

Alternative concept: free-slot stack and floor counters

A scan-based allocation costs O(F×S) in the worst case, where F is the number of floors and S is the maximum slots per floor. Maintaining a stack of free slot coordinates makes allocation and release O(1) amortised, provided each slot is pushed exactly once and duplicate releases are prevented. The stack needs O(total slots) extra references; a free-count array needs O(F) integers. The simple scan uses no additional free-slot structure beyond the occupancy array.

TASK 4 – Design Classes, Objects and Constructors (4 Marks)

Vehicle and ParkingTicket use parameterised constructors so required state is supplied at creation. The this keyword distinguishes fields from parameters. The constructor of ElectricCar calls the Car constructor, which calls Vehicle through super, demonstrating constructor chaining.

```
public abstract class Vehicle {
    private final String registrationNumber;
    protected Vehicle(String registrationNumber, String ownerName) {
        this.registrationNumber = registrationNumber.trim().toUpperCase();
        this.ownerName = ownerName.trim();
    }
}
public class ElectricCar extends Car implements Chargeable {
    public ElectricCar(String regNo, String owner) {
        super(regNo, owner);
    }
}
```

A default constructor is not appropriate for a valid ParkingTicket because a ticket without a vehicle, floor, slot and entry time is invalid. An overloaded constructor may be used for restoring a ticket from persistence. Java does not provide a language-defined copy constructor, but a copy-style constructor can be written if a duplicate representation is needed. A copied ticket should retain the original ticket ID only for archival display; it must not be inserted as a second active ticket.

At runtime, local reference variables are typically held in stack frames, while objects and arrays are allocated on the heap. A ParkingFacility stores references to tickets and vehicles. Passing an object to a method passes its reference value; methods can mutate the referenced object but cannot rebind the caller's local variable.

Alternative concept: setters and static factories

Creation method	Validity	Cost for n tickets
Parameterized constructor	Validates required values before object becomes visible	O(n) time, O(n) object space
Default constructor + setters	Can expose partially initialised or invalid objects; each setter needs validation	O(n) time overall, O(n) space; more calls and failure paths
Static factory method	Can validate, choose subtype or reuse cached instance; constructor can remain private	O(n) typical time and O(n) space; additional method call is constant overhead

All three are asymptotically O(n) for n objects, but constructors and factories provide stronger invariant control. The selected constructor-based approach is easiest to defend for this assignment.

TASK 5 – Apply Static Members and the Object Life Cycle (3 Marks)

ParkingTicket uses a static AtomicLong ticket counter. It belongs to the class because every ticket created in the process must receive a unique identifier. An instance counter would restart for every ParkingTicket object and could issue duplicate IDs. AtomicLong also makes ID generation safe when multiple threads create tickets concurrently.

Static methods are appropriate for operations that do not require an individual object, such as normalising a registration number or validating a constant tariff. A static initialisation block can load immutable configuration once. Static fields should not be used for occupancy when multiple independent facilities exist; occupancy belongs to each ParkingFacility instance.

After a vehicle exits, its ticket is removed from the activeTickets map and the slot is freed. The ticket becomes eligible for garbage collection only when no reachable reference points to it. A report or audit list may keep a reference and extend its lifetime.

Alternative concept: static versus instance ticket counter

For one global campus-wide sequence, a static counter uses O(1) storage regardless of k facility objects and n tickets. An instance counter uses O(k) counter storage and can generate duplicate numbers across facilities unless the facility ID is included in the ticket ID. A robust multi-facility production design could use a globally unique UUID or a composite (facilityCode, localSequence).

TASK 6 – Design the Inheritance Hierarchy (4 Marks)

Vehicle is the superclass. TwoWheeler, Car and DeliveryVan are hierarchical subclasses. ElectricCar extends Car, creating a multilevel chain. ElectricCar implements Chargeable because charging is a capability and not every vehicle must support it.

```
Vehicle (abstract)
 |-- TwoWheeler
 |-- Car
 |    `-- ElectricCar implements Chargeable
 |-- DeliveryVan
 `-- EBike implements Chargeable
```

Common registration and owner fields are private in Vehicle. Subclasses inherit the public getters and protected constructor, then supply their own rate or capability. super(regNo, owner) invokes the parent constructor. A constructor trace for ElectricCar is: Object() → Vehicle(regNo, owner) → Car(regNo, owner) → ElectricCar(regNo, owner), with each superclass constructor completing before the subclass body.

Alternative concept: inheritance versus composition

With inheritance, each subtype overrides its rate behaviour; adding a type generally adds a subclass. With composition, Vehicle would contain a VehicleType or PricingStrategy object that provides the rate. Composition is more flexible when rates vary at runtime and avoids a large subclass tree. Both calculate a fee in O(1) once the strategy/type is available. Inheritance stores subtype-specific state only where required; composition adds a strategy reference per vehicle (O(1) extra space per vehicle).

TASK 7 – Override and Specialise Behaviour (4 Marks)

The getHourlyRate() method is overridden in each concrete vehicle. EBike also overrides calculateFee() because its flat-fee rule is not hourly. DeliveryVan overrides fee calculation to apply its time-window surcharge. Every override uses @Override so the compiler detects signature mistakes.

The final registrationNumber field prevents reassignment after construction. Vehicle.equals() and hashCode() are final and use the normalised registration number, so a Car and ElectricCar with the same registration number compare equal. This matches the rule that one registration identifies one active vehicle, regardless of subtype. toString() supplies a readable diagnostic representation.

Alternative concept: switch-based pricing

A central switch on vehicle type can also compute the fee in O(1) for a fixed number of types. However, every new vehicle type requires editing the switch and often the type enum. With overriding, adding a subclass leaves tested fee callers unchanged. Dynamic dispatch is O(1) in practical terms, while a switch is also O(1) for a fixed finite set; the major difference is extension and maintenance, not asymptotic runtime.

TASK 8 – Apply Polymorphism (4 Marks)

Compile-time polymorphism is method overloading. For example, DeliveryVan has calculateFee(int minutes) and calculateFee(int minutes, int entryHour). The compiler selects the overload from the argument list. Runtime polymorphism occurs when a Vehicle reference refers to a Car, TwoWheeler or ElectricCar object and the overridden method is selected at runtime.

```
Vehicle v = new ElectricCar("TN03C3", "Student"); // upcasting
double fee = v.calculateFee(130);              // ElectricCar/Car rate path
// declared type: Vehicle; actual type: ElectricCar
// selected implementation: inherited Car rate, through Vehicle contract
```

A Vehicle[] can store references to different concrete vehicle objects. The loop invokes calculateFee() without testing each subtype, avoiding long if–else chains. The runtime class of each object determines the implementation.

Completed Code – FeeDemo

The supplied Vehicle hierarchy and FeeDemo are implemented in FeeDemo.java and the model package. The output below follows the expected values.

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

The last line is true because equals() is defined by registration number, not by concrete class. This is correct for active-parking identity: two objects describing the same registered vehicle must be treated as the same vehicle. A stricter domain could additionally reject a subtype mismatch when updating vehicle details.

Alternative concept: instanceof and casting

An instanceof chain performs up to k type checks for k vehicle categories, so its worst-case dispatch check is O(k). Virtual method dispatch is effectively O(1) for the call and keeps the caller independent of subtype count. A switch or instanceof chain also grows in code size and must be edited whenever a new category is added.

TASK 9 – Introduce Abstraction (4 Marks)

Vehicle is abstract because a generic vehicle has no meaningful universal hourly rate. Concrete subclasses implement getHourlyRate(). Chargeable is an interface because charging is an optional capability shared by ElectricCar and EBike, while the two classes remain in different inheritance branches. Java permits a class to extend one superclass and implement multiple interfaces.

A default method in Chargeable could provide a standard charging-session validation or receipt format. A static interface method could validate a common energy input. The current interface keeps only the capability contract, while the implementation supplies the charging rate.

Alternative concept: swap abstract class and interface

If Vehicle became an interface, it could declare default methods but could not hold ordinary per-object instance fields with constructor initialisation. Every implementing class would need to duplicate or delegate registration/owner state. If Chargeable became an abstract class, an ElectricCar could not extend both Car and Chargeable because Java has single class inheritance. Runtime method-call complexity remains effectively constant; object layout may gain extra delegated references if composition is used.

TASK 10 – Enforce Encapsulation through Packages and Access Modifiers (3 Marks)

Package	Classes	Purpose
parking.model	Vehicle, Car, TwoWheeler, ElectricCar, DeliveryVan, EBike, ParkingTicket, Pass types, Chargeable	Domain objects and contracts.
parking.pricing	PricingRule, StandardPricingRule	Replaceable pricing policy.
parking.service	ParkingFacility	Allocation and ticket lifecycle.
parking.app	SmartParkingApp, FeeDemo, Task03ParkingFacility	Runnable entry points and demonstrations.

Modifier	Usage in design	Reason
private	Vehicle fields, ticket fields, occupancy arrays, ticket map	Prevents direct external mutation.
protected	Vehicle constructor and abstract rate method	Allows subclasses to initialise and specialise safely.
public	Validated constructors and public service methods	Defines supported API.
default/package-private	Can be used for internal helpers when no external access is required	Limits implementation details to a package.

Getters expose read-only state. Constructors and methods reject null identifiers, blank names, negative duration, negative energy and invalid floor/slot coordinates. Wrapper classes such as Integer may be used in collections; autoboxing converts int to Integer. Integer objects must be compared by value using equals() or unboxing, not ==, because == may compare object identity outside the cached range.

Alternative concept: public fields

Public mutable fields would allow invalid values such as negative duration, a blank registration number or occupancy greater than capacity. Validation would be repeated at every caller, creating duplicated checks and inconsistent behaviour. A validating setter or constructor performs O(1) work per update, just as caller-side validation does, but centralises the rule in one location and reduces the number of code paths that must be maintained.

TASK 11 – Implement, Test and Defend the Design (5 Marks)

The complete implementation is included in the accompanying ZIP. ParkingFacility uses a jagged boolean array for physical occupancy, a LinkedHashMap for active tickets and a PricingRule reference for tariff calculation. Entry rejects duplicate active registrations, scans for the first free slot, creates a ticket and increments occupancy. Exit finds the ticket by registration in expected O(1) HashMap time, computes the fee, releases the physical slot and removes the ticket.

Validation test cases

Test case	Expected result	Observed result	Status
Allocate until all slots are occupied	Last available slot allocated; next returns null	Confirmed in Task03 output	PASS
Release an occupied slot twice	First true; second false	true then false	PASS
Invalid floor/slot	false; no array exception	false	PASS
Zero-minute stay	₹0	₹0	PASS
61-minute car stay	2 × ₹30 = ₹60	₹60	PASS
Duplicate active registration	Reject with exception	Rejected	PASS
Electric car: 130 min + 10 kWh	₹90 + ₹120 = ₹210	₹210	PASS

Valid one-day pass	Parking fee ₹0	₹0	PASS
E-bike non-zero duration	Flat ₹25	₹25	PASS
Delivery van entered at 11:00 for 2 hours	2 × ₹30 × 2 = ₹120	₹120	PASS

Hand-calculated daily collection example

Consider three fee-paying vehicles: a two-wheeler parked for 61 minutes (2 × ₹10 = ₹20), a car parked for 130 minutes (3 × ₹30 = ₹90), and an electric car parked for 130 minutes plus 10 kWh charging (3 × ₹30 + 10 × ₹12 = ₹210). The expected collection is ₹20 + ₹90 + ₹210 = ₹320. A valid pass vehicle contributes ₹0 parking fee and is excluded from this total.

Complexity of exit lookup: a scan through an array of n active tickets costs O(n); ArrayList lookup by registration is also O(n). A HashMap keyed by registration offers expected O(1) lookup and O(n) storage. Slot allocation by scanning is O(T), where T is the total number of slots. The occupancy array itself uses O(T) space.

Evaluation and limitation

The delivery-van implementation applies the double rate to each billable hour block whose start time falls from 09:00 through 16:59; a block beginning at 17:00 returns to the normal car rate. A production system should also persist tickets and handle payment failures transactionally.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

The changed requirements add e-bikes, delivery vans, one-day passes and five floors with unequal slot counts. A fixed rectangular array is no longer suitable. The jagged array already supports unequal capacities. The Vehicle hierarchy needs new concrete classes; the pass model needs a one-day implementation; pricing must support new rules without modifying the allocation algorithm.

B. Evaluation of the Original Design (2 Marks)

The original abstract Vehicle, Chargeable interface, PricingRule contract and ParkingFacility boundary remain appropriate. The new vehicle categories can be added as subclasses or capability implementations. The tariff logic is not spread through slot allocation; it is delegated to PricingRule and vehicle behaviour. The original allocation and ticket classes can remain unchanged. New categories require new classes and a small extension to pricing policy where the new rule is not naturally vehicle-specific.

C. Adaptation or Redesign (2 Marks)

Decision: MODIFY the original design, not fully redesign it. Retain Vehicle, ParkingTicket, ParkingFacility and the pricing abstraction. Add EBike, DeliveryVan, Pass, MonthlyPass and OneDayPass. Configure the five-floor jagged layout as {20, 50, 50, 75, 100}. E-bike parking is a flat ₹25 per non-zero stay. Delivery vans pay twice the car rate when entering during 09:00–17:00. A valid one-day pass waives the parking fee on its valid date.

D. Evidence-Based Defence (2 Marks)

Evidence	Result / justification
Existing classes changed	Core Vehicle contract and ParkingFacility remain stable; additions are new concrete types and pass classes. PricingRule implementation is the single policy extension point.
Adding a future type	Add a Vehicle subclass and implement optional interfaces such as Chargeable; the parking allocation loop is unchanged.
E-bike test	A non-zero parking duration returns flat ₹25.
Delivery-van test	Two billable hours at double ₹30/hour returns ₹120 for entry at 11:00.
One-day pass test	Valid date returns ₹0 parking charge.
Unequal floors	Jagged array supports independent lengths without wasted placeholder slots.

Trade-off: inheritance is clear for stable vehicle categories, while composition through PricingRule is more flexible for frequently changing tariffs. The modified design balances both: subtype behaviour models vehicle identity and a replaceable pricing interface isolates policy.

Final Conclusion

The campus smart-parking system is modelled using Java object-oriented principles and validated with runnable programs. The design uses a jagged array for floor capacities, a registration-indexed map for ticket lookup, constructor validation, static ticket numbering, inheritance, overriding, polymorphism, abstract classes, interfaces and encapsulated packages. The Day-4 requirements are handled by modifying the original design through extension rather than rewriting the stable facility logic.

APPENDIX A – Source Code and Execution

All Java source files, input configuration, README instructions, build logs and captured outputs are supplied in the companion ZIP archive. The primary application entry point is parking.app.SmartParkingApp. The two required completed-code demonstrations are Task03ParkingFacility and FeeDemo.

Compile:
javac -d out <all Java source files>
Run:
java -cp out parking.app.SmartParkingApp
java -cp out Task03ParkingFacility
java -cp out FeeDemo